

CONVOLUTIONAL NEURAL NETWORK (CNN)

1. From Neural Network to CNN

In the previous sessions, we learned how a Neural Network works and how to build one using **TensorFlow and Keras**.

Today, we will learn a special type of Neural Network mainly used for **images**.

CNN = Convolutional Neural Network

2. Why Do We Need CNN?

An image contains thousands of pixels.

If we directly give all these pixels to a normal Neural Network, the number of connections can become very large.

Also, a normal Neural Network does not naturally focus on the relationship between nearby pixels.

CNN is designed to automatically learn important visual patterns from images.

Example

Cat Image → Edges → Shapes → Eyes/Ears → Face → CAT

3. ANN vs CNN

ANN

Input Features → Dense Layers → Output

Generally useful for structured data such as:

Salary, Marks, House Price, Customer Data

CNN

Image → Convolution → Feature Extraction → Pooling → Output

Mainly used for:

Image Classification, Face Recognition, Object Detection

Simple Difference

ANN → Works with general input features

CNN → Specially designed to learn features from images

4. How Does CNN Understand an Image?

For a computer, an image is a collection of **pixel values**.

CNN does not try to understand the entire image at once.

Instead, it looks at **small regions of the image** and searches for useful patterns.

These patterns can be:

Edges → Shapes → Textures → Objects

5. Convolution

Convolution is the main operation in CNN.

A small matrix called a **Filter or Kernel** moves across different parts of the image.

It performs calculations with each small region and produces values.

Simple Flow

Image



Small Region



Filter



Feature Value



Next Region

The process continues across the image.

6. Filter / Kernel

A **Filter (Kernel)** is a small matrix used to detect patterns in an image.

For example, a filter may learn to detect:

- Vertical edges
- Horizontal edges
- Corners
- Textures

Different filters learn different patterns.

Important

The CNN **learns the useful filter values automatically during training**.

We do not manually tell the model which filter should detect a cat's ear or eye.

7. Feature Map

When a filter scans an image, it produces a new output called a:

Feature Map

A feature map shows where a particular pattern was detected.

Example

Image

↓

Edge Filter

↓

Edge Feature Map

So we can think of a feature map as:

A representation showing where a particular feature is present in the image.

8. ReLU

After convolution, we commonly use the **ReLU activation function**.

ReLU = Rectified Linear Unit

Formula:

$$\text{ReLU}(x) = \max(0, x)$$

Examples:

$$-5 \rightarrow 0$$

$$3 \rightarrow 3$$

$$8 \rightarrow 8$$

ReLU introduces **non-linearity**, allowing the network to learn more complex patterns.

9. Pooling

After convolution, feature maps can be large.

Pooling reduces their size while keeping important information.

The two common types are:

Max Pooling

Average Pooling

10. Max Pooling

Max Pooling selects the **largest value** from a small region.

Example:

[1 5]

[3 2]

Maximum value:

5

Therefore:

Max Pooling → 5

It keeps the strongest detected feature from that region.

11. Average Pooling

Average Pooling calculates the average of the values.

Example:

[1 5]

[3 2]

Average:

$$(1 + 5 + 3 + 2) / 4 = 2.75$$

Therefore:

Average Pooling → 2.75

Remember

Max Pooling → Maximum value

Average Pooling → Average value

12. Why Pooling?

Pooling helps to:

- Reduce the size of feature maps
 - Reduce computation
 - Keep important information
 - Make the model less sensitive to small changes in position
-

13. How CNN Learns Features

CNN learns features gradually.

Early Layers

Edges → Lines → Corners

Middle Layers

Shapes → Textures → Eyes → Ears

Deeper Layers

Face → Object Parts → Complete Object

So:

Simple Features → Complex Features

14. Flatten

After convolution and pooling, we have feature maps.

Before sending them to a Dense layer, we convert them into a single long vector.

This is called:

Flattening

Flow

Feature Maps

↓

Flatten

↓

One Long Vector

15. Fully Connected Layer

After extracting important features, the CNN uses a **Fully Connected/Dense Layer** to make the final decision.

For example:

Edges + Shapes + Eyes + Ears + Face Features

↓

Dense Layer

↓

Prediction

16. Output Layer

The final layer produces the prediction.

Example:

Cat → 0.90

Dog → 0.07

Horse → 0.03

The highest probability is:

Cat

Therefore:

Prediction = CAT

17. Complete CNN Architecture

IMAGE



CONVOLUTION



RELU



POOLING



CONVOLUTION



RELU



POOLING



FLATTEN



DENSE LAYER



OUTPUT

In Simple Words

Extract Features → Reduce Size → Classify

18. CNN Example – Cat Recognition

Suppose we provide a Cat Image.

CAT IMAGE



Edges



Shapes



Eyes / Ears / Nose



Face Features



Dense Layer



CAT

The important idea is:

CNN first learns useful features from the image and then uses those features to make the prediction.

19. Real-World Applications

CNNs are commonly used for:

Face Recognition

Medical Image Analysis

OCR / Text Recognition

Object Detection

Self-Driving Cars

Image Classification

20. ANN vs CNN – Quick Revision

ANN

General-purpose Neural Network

Uses Dense layers

Works well with structured data

Does not naturally extract visual features

CNN

Mainly designed for images

Uses Convolution + Pooling + Dense

Works well with image/spatial data

Automatically learns visual features

FINAL CONCEPT

ANN

Input → Dense Layers → Output

CNN

Image → Convolution → Feature Extraction → Pooling → Dense → Output

Remember:

CNN = Automatically learns visual features from images.

The complete idea:

Image → Edges → Shapes → Features → Object → Prediction

FINAL DEEP LEARNING REVISION

We learned:

Neural Network Fundamentals

↓

Building Neural Networks using TensorFlow/Keras

↓

CNN – Convolutional Neural Network

The main idea of Deep Learning is:

Neural Networks can automatically learn useful patterns from data.

And CNN is a specialized Neural Network that is particularly effective at learning **visual patterns from images**.